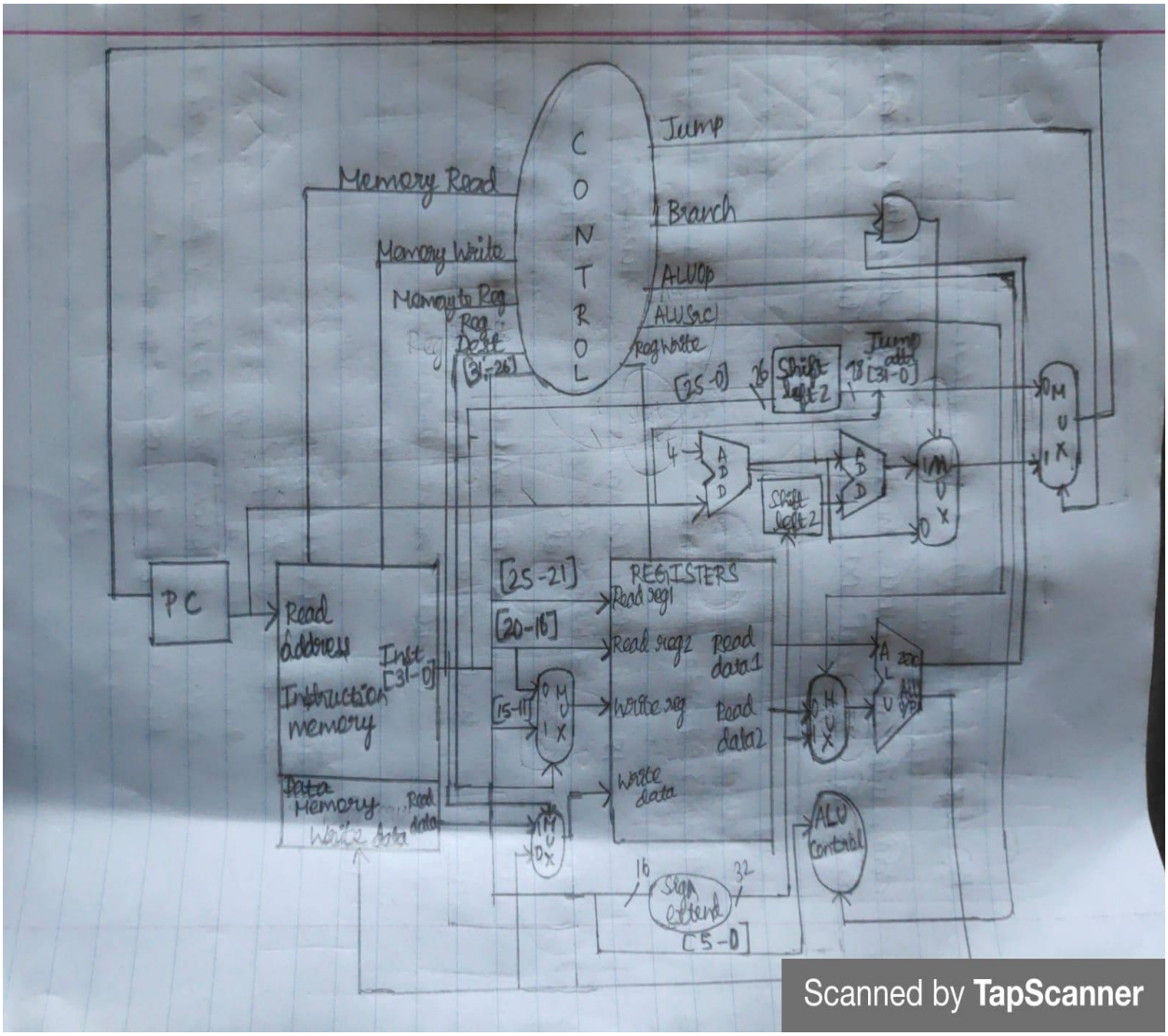
Computer Architecture (ECGR 4181/5181)

Assignment 3: RISC-V Decoder Design

Consider the Decoder of a processor using the von Neumann architecture (discussed in class). The processor uses a subset of the RISC-V ISA (RV32I, RV32F and RV32M). The double words, ecall, ebreak, fence, and “cs” ops are to be ignored. We are considering UNPIPELINED design for this assignment.

1) Draw the schematic of the architecture where the different components (in the fetch, decode, execute, writeback stages) are shown as black boxes, their ports are clearly defined, and connections between these ports detailed. Explain the purpose of these ports and connections.



2) Write the Decoder code (in C++) and comment in the code to highlight the executions for every instruction.

```
#include <iostream>
#include <bitset>
#include <string>
#include <vector>
using namespace std;


enum ControlLines {
    ALU_OP,
    REG_DEST,
    REG_WRITE,
    ALU_SRC,
    JUMP,
    BRANCH,
    MEM_READ,
    MEM_WRITE,
    MEM_TO_REG,
    NUM_CONTROLS
};


void printControlLines(const string& type, const string& instruction, const vector<bool>&
controls) {
    cout << "Instruction Type: " << type << "\n"
        << instruction << "\n"
        << "ALU op : " << controls[ALU_OP] << "\n"
        << "Reg dest : " << controls[REG_DEST] << "\n"
        << "Reg write : " << controls[REG_WRITE] << "\n"
        << "ALU src : " << controls[ALU_SRC] << "\n"
        << "Jump : " << controls[JUMP] << "\n"
```

```
        << "Branch : " << controls[BRANCH] << "\n"
        << "Memory read : " << controls[MEM_READ] << "\n"
        << "Memory write : " << controls[MEM_WRITE] << "\n"
        << "Memory to register : " << controls[MEM_TO_REG] << "\n\n";
}

void decodeRType(uint32_t input) {
    vector<bool> controls(NUM_CONTROLS, 0);
    controls[ALU_OP] = 1;
    controls[REG_DEST] = 1;
    controls[REG_WRITE] = 1;

    uint32_t funct3 = (input >> 12) & 0x7;
    uint32_t funct7 = (input >> 25) & 0x7F;
    uint32_t rd = (input >> 7) & 0x1F;
    uint32_t rs1 = (input >> 15) & 0x1F;
    uint32_t rs2 = (input >> 20) & 0x1F;

    string instruction;

    switch (funct3) {
        case 0b000:
            switch (funct7) {
                case 0b0000000:
                    instruction = "ADD r" + to_string(rd) + " r" + to_string(rs1) + " r" + to_string(rs2);
                    break;
                case 0b0100000:
                    instruction = "SUB r" + to_string(rd) + " r" + to_string(rs1) + " r" + to_string(rs2);
                    break;
                case 0b0000001:
```

```
                instruction = "MUL r" + to_string(rd) + " r" + to_string(rs1) + " r" + to_string(rs2);
                break;
            }
            break;
        case 0b001:

            switch(funct7){
                case 0b0000001:
                        instruction = "MULH r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
               case 0b0000000:
                instruction = "SLL r" + to_string(rd) + " r" + to_string(rs1) + " r" + to_string(rs2);
                break;

                  }
            case 0b010:
                  switch(funct7){
                     case 0b0000001:
                        instruction = "MULHSU r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                     case 0b0000000:
                        instruction = "SLT r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                     }
            case 0b011:
                  switch(funct7){
                     case 0b0000001:
```

```
                    instruction = "MULHSU r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
            break;
                case 0b0000000:
                    instruction = "SLTU r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
            break;
                    }
        case 0b100:
            switch(funct7){
                case 0b0000001:
                    instruction = "DIV r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
            break;
                case 0b0000000:
                    instruction = "XOR r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
            break;
                    }
        case 0b110:
            switch(funct7){
                case 0b0000001:
                    instruction = "OR r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
            break;
                case 0b0000000:
                    instruction = "REM r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
            break;
                    }
        case 0b111:
            switch(funct7){
```

```
                    case 0b0000001:
                        instruction = "AND r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                    case 0b0000000:
                        instruction = "REMU r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                        }

        case 0b101:
            switch(funct7){
                    case 0b0000001:
                        instruction = "SRL r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                    case 0b0000000:
                        instruction = "DIVU r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                    case 0b0100000:
                        instruction = "SRA r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                        }
        break;
        default :
            cout<<"Error"<<endl;
}


   printControlLines("R-Type", instruction, controls);
```

```
}
void decodeRType_Word(uint32_t input) {
    vector<bool> controls(NUM_CONTROLS, 0);
    controls[ALU_OP] = 1;
    controls[REG_DEST] = 1;
    controls[REG_WRITE] = 1;

    uint32_t funct3 = (input >> 12) & 0x7;
    uint32_t funct7 = (input >> 25) & 0x7F;
    uint32_t rd = (input >> 7) & 0x1F;
    uint32_t rs1 = (input >> 15) & 0x1F;
    uint32_t rs2 = (input >> 20) & 0x1F;

    string instruction;

    switch (funct3) {
        case 0b000:
            switch (funct7) {
                case 0b0000000:
                    instruction = "ADDW r" + to_string(rd) + " r" + to_string(rs1) + " r" + to_string(rs2);
                    break;
                case 0b0100000:
                    instruction = "SUBW r" + to_string(rd) + " r" + to_string(rs1) + " r" + to_string(rs2);
                    break;
                case 0b0000001:
                    instruction = "MULW r" + to_string(rd) + " r" + to_string(rs1) + " r" + to_string(rs2);
                    break;
            }
            break;
        case 0b001:
```

```
            switch(funct7){

             case 0b0000000:
                instruction = "SLLW r" + to_string(rd) + " r" + to_string(rs1) + " r" + to_string(rs2);
                break;

                }

        case 0b100:
                switch(funct7){
                    case 0b0000001:
                            instruction = "DIVW r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                            }
        case 0b101:
                switch(funct7){
                    case 0b0000000:
                            instruction = "SRLW r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                    case 0b0000001:
                            instruction = "DIVUW r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                    case 0b0100000:
                            instruction = "SRAW r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
```

```
                        }
        case 0b110:
                switch(funct7){
                    case 0b0000001:
                        instruction = "REMW r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;
                        }

        case 0b111:
                switch(funct7){
                    case 0b0000001:
                        instruction = "REMUW r" + to_string(rd) + " r" + to_string(rs1) + " r" +
to_string(rs2);
                break;

                        }
        break;
        default :
                cout<<"Error"<<endl;
}



    printControlLines("R-Type", instruction, controls);
}
void decodeIType(uint32_t input) {
    vector<bool> controls(NUM_CONTROLS, 0);

    uint32_t opcode = input & 0x7F;
    uint32_t funct3 = (input >> 12) & 0x7;
    uint32_t funct7 = (input >> 25) & 0x7F;
```

```
uint32_t rd = (input >> 7) & 0x1F;
uint32_t rs1 = (input >> 15) & 0x1F;
int32_t imm = static_cast<int32_t>(input) >> 20;  // signed immediate

string instruction;
controls[ALU_OP] = 1;
controls[REG_WRITE] = 1;
controls[REG_DEST] = 1;

if (opcode == 0b0010011) {  // I-type ALU
    controls[ALU_SRC] = 1;
    switch (funct3) {
        case 0b000:
            instruction = "ADDI r" + to_string(rd) + ", r" + to_string(rs1) + ", " + to_string(imm);
            break;

        case 0b010:
            instruction = "SLTI r" + to_string(rd) + ", r" + to_string(rs1) + ", " + to_string(imm);
            break;
        case 0b011:
            instruction = "SLTIU r" + to_string(rd) + ", r" + to_string(rs1) + ", " + to_string(imm);
            break;
        case 0b100:
            instruction = "XORI r" + to_string(rd) + ", r" + to_string(rs1) + ", " + to_string(imm);
            break;
        case 0b110:
            instruction = "ORI r" + to_string(rd) + ", r" + to_string(rs1) + ", " + to_string(imm);
            break;
        case 0b111:
            instruction = "ANDI r" + to_string(rd) + ", r" + to_string(rs1) + ", " + to_string(imm);
```

```
            break;
        case 0b001:
            instruction = "SLLI r" + to_string(rd) + ", r" + to_string(rs1) + ", " + to_string(imm);
            break;
        case 0b101:
                switch(funct7){
                        case 0b0000000:
                        instruction = "SRLI r" + to_string(rd) + ", r" + to_string(rs1) + ", " +
to_string(imm);
                        break;
                        case 0b0100000:
                        instruction = "SRAI r" + to_string(rd) + ", r" + to_string(rs1) + ", " +
to_string(imm);
                        break;
                        }
             default :
                cout<<"Error"<<endl;
}
    printControlLines("I-Type", instruction, controls);
  }
else if (opcode == 0b0011011) {
    controls[ALU_SRC] = 1;
    switch (funct3) {
        case 0b000:
            instruction = "ADDI r" + to_string(rd) + ", r" + to_string(rs1) + ", " + to_string(imm);
            break;

        case 0b001:
                switch(funct7){
                        case 0b0000000:
```

```
                        instruction = "SLLIW r" + to_string(rd) + ", r" + to_string(rs1) + ", " +
to_string(imm);
                        break;}

        case 0b101:
                switch(funct7){
                        case 0b0000000:
                        instruction = "SRLIW r" + to_string(rd) + ", r" + to_string(rs1) + ", " +
to_string(imm);
                        break;
                        case 0b0100000:
                        instruction = "SRAIW r" + to_string(rd) + ", r" + to_string(rs1) + ", " +
to_string(imm);
                        break;
                        }
            default :
                cout<<"Error"<<endl;
                printControlLines("I-Type", instruction, controls);
}
}
else if (opcode == 0b0000011) {  // Load instructions
            controls[ALU_OP] = 1;
           controls[MEM_READ] = 1;
       controls[ALU_SRC] = 1;
       switch (funct3) {
           case 0b000:
              instruction = "LB r" + to_string(rd) + ", " + to_string(imm) + "(r" + to_string(rs1) + ")";
              break;
            case 0b001:
              instruction = "LH r" + to_string(rd) + ", " + to_string(imm) + "(r" + to_string(rs1) + ")";
              break;
```

```
            case 0b010:
               instruction = "LW r" + to_string(rd) + ", " + to_string(imm) + "(r" + to_string(rs1) + ")";
               break;
            case 0b100:
               instruction = "LBU r" + to_string(rd) + ", " + to_string(imm) + "(r" + to_string(rs1) + ")";
               break;
            case 0b101:
               instruction = "LHU r" + to_string(rd) + ", " + to_string(imm) + "(r" + to_string(rs1) + ")";
               break;
                default :
                      cout<<"Error"<<endl;

}
 printControlLines("I-Type", instruction, controls);
}}
void decodeSType(uint32_t input) {
    vector<bool> controls(NUM_CONTROLS, 0);

    uint32_t funct3 = (input >> 12) & 0x7;
    uint32_t rs1 = (input >> 15) & 0x1F;
    uint32_t rs2 = (input >> 20) & 0x1F;

    // Constructing the 12-bit signed immediate for S-type
    int32_t imm = ((input & 0xFE000000) >> 20) | ((input >> 7) & 0x1F);  // 12-bit signed
immediate for S-type
    if (input & 0x80000000) {  // Sign extension
        imm |= 0xFFFFF000;
    }

    string instruction;
```

```
    controls[ALU_OP] = 1;
    controls[MEM_WRITE] = 1;  // Set for all store instructions
    controls[ALU_SRC] = 1;   // To compute effective address

    switch (funct3) {
        case 0b000:
            instruction = "SB r" + to_string(rs2) + ", " + to_string(imm) + "(r" + to_string(rs1) + ")";
            break;
        case 0b001:
            instruction = "SH r" + to_string(rs2) + ", " + to_string(imm) + "(r" + to_string(rs1) + ")";
            break;
        case 0b010:
            instruction = "SW r" + to_string(rs2) + ", " + to_string(imm) + "(r" + to_string(rs1) + ")";
            break;

    }

    printControlLines("S-Type", instruction, controls);
}
void decodeSBType(uint32_t input) {
    vector<bool> controls(NUM_CONTROLS, 0);

    uint32_t funct3 = (input >> 12) & 0x7;
    uint32_t rs1 = (input >> 15) & 0x1F;
    uint32_t rs2 = (input >> 20) & 0x1F;

    controls[ALU_OP] = 1;  // Assuming ALU_OP = 1 signifies branch comparison
    controls[BRANCH] = 1; // Set the control signal for branching

    string instruction;
```

```
    switch (funct3) {
        case 0b000:
            instruction = "BEQ r" + to_string(rs1) + ", r" + to_string(rs2);
            break;
        case 0b001:
            instruction = "BNE r" + to_string(rs1) + ", r" + to_string(rs2);
            break;
        case 0b100:
            instruction = "BLT r" + to_string(rs1) + ", r" + to_string(rs2);
            break;
        case 0b101:
            instruction = "BGE r" + to_string(rs1) + ", r" + to_string(rs2);
            break;
        case 0b110:
            instruction = "BLTU r" + to_string(rs1) + ", r" + to_string(rs2);
            break;
        case 0b111:
            instruction = "BGEU r" + to_string(rs1) + ", r" + to_string(rs2);
            break;
            default:
                    cout<<"Error"<<endl;
    }

    printControlLines("SB-Type", instruction, controls);
}
void decodeUJType(uint32_t input, uint32_t opcode) {
    vector<bool> controls(NUM_CONTROLS, 0);
    uint32_t rd = (input >> 7) & 0x1F;
    uint32_t rs1 = (input >> 15) & 0x1F;
```

```
    string instruction;

    if(opcode == 0b1100111) {
        uint32_t funct3 = (input >> 12) & 0x7;
        if(funct3 == 0b000) {
            instruction = "JALR r" + to_string(rd) + ", r" + to_string(rs1) + ", imm";
            controls[ALU_OP] = 1;
            controls[ALU_SRC] = 1;
            controls[REG_WRITE] = 1;
            controls[JUMP] = 1;
        }
        printControlLines("UJ-Type", instruction, controls);
    } else if(opcode == 0b1101111) {
        instruction = "JAL r" + to_string(rd) + ", imm";
        controls[ALU_SRC] = 1;
        controls[REG_WRITE] = 1;
        controls[JUMP] = 1;
    }

    printControlLines("UJ-Type", instruction, controls);;
}
void decodeUType(uint32_t input, uint32_t opcode) {
    vector<bool> controls(NUM_CONTROLS, 0);
    uint32_t rd = (input >> 7) & 0x1F;
    uint32_t funct3 = (input >> 12) & 0x7;

    string instruction;

    if(opcode == 0b0110111 && funct3 == 0b000) {
```

```
        instruction = "LUI r" + to_string(rd) + ", imm";
        controls[REG_DEST] = 1;
        controls[REG_WRITE] = 1;
        printControlLines("U-Type", instruction, controls);
    } else if(opcode == 0b0010111 && funct3 == 0b000) {
        instruction = "AUIPC r" + to_string(rd) + ", imm";
        controls[ALU_OP] = 1;
        controls[ALU_SRC] = 1;
        controls[REG_DEST] = 1;
        controls[REG_WRITE] = 1;
    }

    printControlLines("U-Type", instruction, controls);
}


int main() {
    char cont;
    do {
        string Input_binaryFormat;
        cout << "Enter 32-bit binary input: ";
        cin >> Input_binaryFormat;

        uint32_t input = stoul(Input_binaryFormat, nullptr, 2);
        uint32_t opcode = input & 0x7F;

        switch (opcode) {
            case 0b0110011:  // R-type
                decodeRType(input);
                break;
```

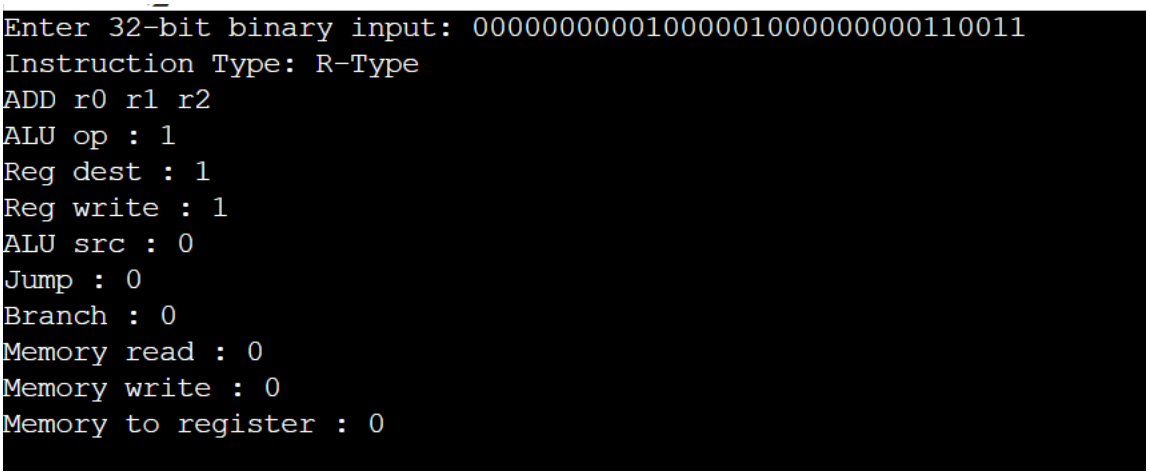
```
case 0b0111011: //R-type Word
            decodeRType_Word(input);
            break;
case 0b0010011:  // I-type
    decodeIType(input);
    break;
case 0b0011011:  // I-type
    decodeIType(input);
    break;
case 0b0000011:  // I-type
    decodeIType(input);
    break;
case 0b0100011:  // S-type
    decodeSType(input);
    break;
case 0b1100011:  // SB-type
    decodeSBType(input);
    break;

case 0b1100111:  // UJ-type
    decodeUJType(input,opcode);
    break;
case 0b1101111:  // UJ-type
    decodeUJType(input,opcode);
    break;
case 0b0110111:  // U-type
    decodeUType(input,opcode);
    break;
case 0b0010111:  // U-type
    decodeUType(input,opcode);
```

```
            break;
        }

        cout<<"Do you want to continue?(Y/N)";
cin>>cont;
}while(cont == 'Y' || cont == 'y');
return 0;
    }
```

OUTPUT



```
Do you want to continue?(Y/N)y
Enter 32-bit binary input: 00000000001000001000000110010011
Instruction Type: I-Type
ADDI r3, r1, 2
ALU op : 1
Reg dest : 1
Reg write : 1
ALU src : 1
Jump : 0
Branch : 0
Memory read : 0
Memory write : 0
Memory to register : 0
```

```
Do you want to continue?(Y/N)y
Enter 32-bit binary input: 00000011110001010000001010000011
Instruction Type: I-Type
LB r5, 60(r10)
ALU op : 1
Reg dest : 1
Reg write : 1
ALU src : 1
Jump : 0
Branch : 0
Memory read : 1
Memory write : 0
Memory to register : 0
```

```
Do you want to continue?(Y/N)y
Enter 32-bit binary input: 10101011010101010010101000100011
Instruction Type: S-Type
SW r21, -1356(r10)
ALU op : 1
Reg dest : 0
Reg write : 0
ALU src : 1
Jump : 0
Branch : 0
Memory read : 0
Memory write : 1
Memory to register : 0
```

```
Do you want to continue?(Y/N)y
Enter 32-bit binary input: 11010101010101010000101011100011
Instruction Type: SB-Type
BEQ r10, r21
ALU op : 1
Reg dest : 0
Reg write : 0
ALU src : 0
Jump : 0
Branch : 1
Memory read : 0
Memory write : 0
Memory to register : 0
```

```
Enter 32-bit binary input: 00000001010011111111010101101111
Instruction Type: UJ-Type
JAL r10, imm
ALU op : 0
Reg dest : 0
Reg write : 1
ALU src : 1
Jump : 1
Branch : 0
Memory read : 0
Memory write : 0
Memory to register : 0
```

```
Enter 32-bit binary input: 00000010101001010000101011100111
Instruction Type: UJ-Type
JALR r21, r10, imm
ALU op : 1
Reg dest : 0
Reg write : 1
ALU src : 1
Jump : 1
Branch : 0
Memory read : 0
Memory write : 0
Memory to register : 0
```

Submitted by,

Poojitha Rajapuram,

Indhuja Gudluru